Experiment No.09

Title: Data Clustering using K-Means Algorithm

Tools Required:

Python (Jupyter Notebook / Google Colab)

Libraries: numpy, matplotlib, scikit-learn, seaborn

Objective:

To understand and implement the K-Means Clustering algorithm for unsupervised classification of data into clusters based on feature similarity.

Theory Overview:

K-Means is an unsupervised learning algorithm that groups similar data points into K clusters.

It works by:

Randomly initializing K cluster centroids.
Assigning each data point to the nearest centroid.
Updating centroids as the mean of assigned points.
Repeating steps 2–3 until convergence.
It minimizes the within-cluster sum of squares (WCSS).

Experiment Tasks:

Step 1: Load a Dataset

Use a simple dataset for visualization like the Iris dataset (only 2 features for plotting).

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
X = pd.DataFrame(iris.data[:, :2], columns=iris.feature_names[:2])  # Using only 2 features.
```

Step 2: Visualize the Data

```
import matplotlib.pyplot as plt


plt.scatter(X.iloc[:, 0], X.iloc[:, 1], s=50)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
```

```
plt.title('Raw Data Visualization')
plt.show()
```

Step 3: Apply K-Means Clustering

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
```

Step 4: Visualize Clusters and Centroids

```
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X',
label='Centroids')
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('K-Means Clustering Result')
plt.legend()
plt.show()
```

Step 5: Evaluate Clustering Performance (Optional)

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, y_kmeans)
print("Silhouette Score:", score)
```

Observations to Record:

Final centroid coordinates.

Number of iterations to converge.

Silhouette Score (if applicable).

Comparison with ground truth (if labels are available, e.g., Iris dataset).

Conclusion:

Experiment No.8

Title: Maximum Likelihood Estimation (MLE) for Learning a Classifier

Tools Required:

Python (Jupyter Notebook / Google Colab)

Libraries: numpy, scikit-learn, matplotlib, pandas

Objective:

To understand and implement Maximum Likelihood Estimation (MLE) to train a classifier (Logistic Regression) from data and evaluate its performance.

Theory Overview:

Maximum Likelihood Estimation (MLE) is a fundamental statistical method used to estimate the parameters of a probabilistic model. In the context of **learning a classifier**, MLE helps us find the parameters of the model that make the observed training data most probable. Here's a brief explanation:

1. Objective of MLE in Classification

Given a dataset D={(x1,y1),(x2,y2),…,(xn,yn)}D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}D={(x1,y1),(x2,y2),…,(xn,yn)}, where:

- xix_ixi are input features,
- yiy_iyi are the corresponding class labels,
- and the classifier defines a **conditional probability model** P(y|x;θ)P(y | x; \theta)P(y|x;θ), parameterized by θ\thetaθ,

MLE aims to find the parameter θ∗\theta^*θ∗ that maximizes the likelihood of the observed labels given the inputs.

2. Likelihood Function

The **likelihood** of the data under the model is:

L(θ)=∏i=1nP(yi|xi;θ)L(\theta) = \prod_{i=1}^n P(y_i | x_i; \theta)L(θ)=i=1∏nP(yi|xi;θ)

Since products of probabilities can become very small and hard to work with, we usually take the **log-likelihood**:

log⁡L(θ)=∑i=1nlog⁡P(yi|xi;θ)\log L(\theta) = \sum_{i=1}^n \log P(y_i | x_i; \theta)logL(θ)=i=1∑nlogP(yi|xi;θ)

3. Maximum Likelihood Estimation

The goal is to **maximize the log-likelihood**:

θ∗=arg⁡max⁡θ∑i=1nlog⁡P(yi|xi;θ)\theta^* = \arg\max_\theta \sum_{i=1}^n \log P(y_i | x_i; \theta)θ∗=argθmaxi=1∑nlogP(yi|xi;θ)

This turns the learning problem into an **optimization problem**.

4. Example: Logistic Regression

In logistic regression for binary classification:

P(y=1|x;θ)=σ(θTx)=11+e−θTxP(y = 1 | x; \theta) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}P(y=1|x;θ)=σ(θTx)=1+e−θTx1

MLE finds the θ\thetaθ that best fits this model to the data by maximizing the log-likelihood of the observed class labels.

Logistic Regression is a commonly used classifier trained using MLE.

Experiment Tasks:

Ste**p 1: Dataset Preparation**

Load a binary classification dataset (e.g., Breast Cancer dataset from sklearn.datasets).

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
data = load_breast_cancer()
X = data.data
y = data.target
```

Step 2: Data Pre-processing

Normalize features and split into train-test sets.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 3: Model Training using MLE (Logistic Regression)

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

MLE is used internally here to estimate the parameters by maximizing the log-likelihood function.

Step 4: Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Step 5: Visualize Probabilities (Optional)

```
import matplotlib.pyplot as plt

probabilities = model.predict_proba(X_test)[:, 1]

plt.hist(probabilities, bins=20, color='skyblue')

plt.title("Predicted Probabilities (MLE)")

plt.xlabel("Probability of Class 1")

plt.ylabel("Frequency")

plt.show()
```

Observations to be record :

1. Accuracy on the test set.
2. Effect of data normalization.
3. How the model probability relates to prediction confidence.
4. Time taken for training and convergence behavior (if using gradient-based methods explicitly).

Conclusion:

Lab Experiment: Principal Component Analysis (PCA) for Pattern Recognition

Objective

To understand and implement Principal Component Analysis (PCA) for dimensionality reduction and pattern recognition in high-dimensional datasets.

Materials Required

Computer with Python installed.

Libraries: NumPy, pandas, matplotlib, scikit-learn.

Dataset: Any high-dimensional dataset (e.g., Iris dataset or a custom dataset).

Theory

Principal Component Analysis (PCA) is a statistical technique used to reduce the dimensionality of data while retaining the maximum variance. It transforms correlated variables into uncorrelated principal components, which are ordered by the amount of variance they capture. PCA is widely used for preprocessing in pattern recognition tasks, such as image classification and signal processing.

Procedure

Step 1: Dataset Preparation

Load the dataset into Python using pandas.

Visualize the dataset to understand its structure and features.

Step 2: Data Preprocessing

Standardize the data to have zero mean and unit variance using StandardScaler from scikit-learn.

Step 3: Apply PCA

Compute the covariance matrix of the standardized data.
Perform eigendecomposition to extract eigenvalues and eigenvectors.
Select the top principal components based on eigenvalues.
Project the data onto the new feature space.

Step 4: Visualization

Plot the first two principal components to visualize patterns in reduced dimensions.

Compare clustering or separability of classes in PCA-transformed space.

Step 5: Pattern Recognition

Use PCA-transformed features as input to a machine learning model (e.g., SVM or k-NN).

Evaluate model performance using metrics like accuracy or F1-score.

Python Code

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Step 1: Load Dataset
data = pd.read_csv('dataset.csv')  # Replace with your dataset file
X = data.iloc[:, :-1].values  # Features
y = data.iloc[:, -1].values   # Labels

# Step 2: Standardize Data
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# Step 3: Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_std)

# Step 4: Visualization
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k')
```

```
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Visualization')
plt.colorbar(label='Class')
plt.show()
```

 # Step 5: Pattern Recognition with SVM

```
X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)
svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
```

 # Evaluate Model Performance

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of SVM on PCA-transformed data: {accuracy * 100:.2f}%")
```

Observations

Examine how PCA reduces dimensionality while preserving variance.

Observe clustering/separation of classes in PCA-transformed space.

Compare model performance before and after applying PCA.

Conclusion

PCA effectively reduces dimensionality and highlights patterns in high-dimensional datasets. It simplifies data visualization and enhances machine learning model performance by mitigating multicollinearity and overfitting.

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 06

(“Classification of Regions of a Classifier Using Discriminant Functions ")

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS: M.Tech

SEMESTER: II

DATE OF SUBMISSION:

Title: Classification of Regions of a Classifier Using Discriminant Functions

Objective

To understand and implement discriminant functions for pattern classification, focusing on binary and multi-class classification problems. Students will learn to define decision regions and boundaries using linear discriminant analysis (LDA) and visualize these regions.

Theory

Discriminant Functions

Discriminant functions are used to classify data points into different categories by partitioning the feature space into decision regions.

- **Binary Classification**: One discriminant function separates two classes.
- **Multi-Class Classification**: Multiple discriminant functions are used to classify data into more than two classes.

Decision Rule

- Assign a data point to the class corresponding to the highest value of the discriminant function.
- For binary classification:
  y(x)=wTx+w0$y(\mathbf{x})=\mathbf{w}^T\mathbf{x}+w0$
  Decision Rule:
  - If y(x)>0$y(\mathbf{x})>0$, assign to Class C1$C1$.
  - If y(x)≤0$y(\mathbf{x})\leq 0$, assign to Class C2$C2$.
- For multi-class classification:
  yk(x)=wkTx+wk0$y_k(\mathbf{x})=\mathbf{w}_k^T\mathbf{x}+w_k0$
  Assign to Class CkCk if yk(x)>yj(x)$y_k(\mathbf{x})>y_j(\mathbf{x})$ for all j≠kj□=k.

Linear Discriminant Analysis (LDA)

LDA assumes Gaussian distributions for each class and uses shared covariance matrices. The discriminant function is linear:

yk(x)=wkTx+wk0$y_k(\mathbf{x})=\mathbf{w}_k^T\mathbf{x}+w_k0$

where

wk=Σ−1μkwk=Σ−1μ_k and

wk0=−12μkTΣ−1μk+ln⁡P(Ck)$w_k0=-21\mu_k^T\Sigma-1\mu_k+\ln P(Ck)$.

Apparatus/Software Required

1. Python programming environment (e.g., Jupyter Notebook, Google Colab).
2. Libraries: numpy, matplotlib, scikit-learn.
3.

Procedure

Step 1: Binary Classification

1. **Generate two classes of data using Gaussian distributions.**

```
import numpy as np
import matplotlib.pyplot as plt


# Generate data for two classes
np.random.seed(0)
mean1, mean2 = [2, 2], [6, 6]
cov = [[1, 0.5], [0.5, 1]]  # Shared covariance matrix

class1 = np.random.multivariate_normal(mean1, cov, 100)
class2 = np.random.multivariate_normal(mean2, cov, 100)

# Plot the data
plt.scatter(class1[:, 0], class1[:, 1], label="Class 1", color="blue")
plt.scatter(class2[:, 0], class2[:, 1], label="Class 2", color="red")
plt.legend()
plt.title("Binary Classification Data")
plt.show()
```

2. Define a linear discriminant function and plot the decision boundary.

```
# Define weights and bias for the linear discriminant function
w = np.array([1, -1])  # Example weight vector
b = -4                 # Example bias

# Plot decision boundary
x_vals = np.linspace(0, 8, 100)
y_vals = (-w[0] * x_vals - b) / w[1]

plt.scatter(class1[:, 0], class1[:, 1], label="Class 1", color="blue")
plt.scatter(class2[:, 0], class2[:, 1], label="Class 2", color="red")
plt.plot(x_vals, y_vals, label="Decision Boundary", color="green")
plt.legend()
plt.title("Binary Classification with Decision Boundary")
plt.show()
```

3. Classify points based on the discriminant function:

python

```
def classify(x):
    return "Class 1" if np.dot(w, x) + b > 0 else "Class 2"

test_point = np.array([4, 4])
print(f"Test Point {test_point} belongs to {classify(test_point)}")
```

Step 2: Multi-Class Classification

1. Generate data for three classes.

python

```
mean3 = [4, -2]

class3 = np.random.multivariate_normal(mean3, cov, 100)

# Plot the data
plt.scatter(class1[:, 0], class1[:, 1], label="Class 1", color="blue")
plt.scatter(class2[:, 0], class2[:, 1], label="Class 2", color="red")
plt.scatter(class3[:, 0], class3[:, 1], label="Class 3", color="green")
plt.legend()
plt.title("Multi-Class Classification Data")
plt.show()
```

2. Train a multi-class classifier using scikit-learn's Linear Discriminant Analysis.

```
python
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Combine data and labels
X = np.vstack((class1, class2, class3))
y = np.array([0]*100 + [1]*100 + [2]*100)  # Labels: Class indices

# Train LDA model
lda = LinearDiscriminantAnalysis()
lda.fit(X, y)

# Plot decision boundaries
from matplotlib.colors import ListedColormap

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))

Z = lda.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.8, cmap=ListedColormap(["blue", "red", "green"]))

# Overlay training points
plt.scatter(X[:100, 0], X[:100, 1], label="Class 1", color="blue")
plt.scatter(X[100:200, 0], X[100:200, 1], label="Class 2", color="red")
plt.scatter(X[200:, 0], X[200:, 1], label="Class 3", color="green")
plt.legend()
plt.title("Multi-Class Classification with Decision Boundaries")
plt.show()
```

Expected Outcomes

1. **Binary Classification**: Visualize the decision boundary and classify test points.
2. **Multi-Class Classification**: Visualize decision boundaries and classify test points among multiple classes.

Result:

Conclusion:

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 05

(“Pattern Recognition with Bayesian Classification")

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS: M.Tech

SEMESTER: II

DATE OF SUBMISSION:

Title:

Pattern Recognition with Bayesian Classification

Aim:

To classify data points using the Naive Bayes classifier and evaluate its performance based on accuracy.

Theory:

Bayes' Theorem is one of the fundamental concepts in probability theory, named after the English statistician Thomas Bayes. It provides a powerful method for calculating conditional probabilities, which are the likelihoods of events based on prior knowledge or evidence. This theorem is particularly useful when new information becomes available, allowing us to update our predictions or beliefs accordingly.

In simple terms, Bayes' Theorem allows us to revise our initial assumptions about the probability of an event as new data is introduced. It connects prior probability, the likelihood of current evidence, and the overall probability of the observed data to give a more accurate estimation of the event in question.

Procedure:

Step-by-Step Procedure

1. Setup Environment

Make sure to install the required libraries if you haven't already:

bash

pip install numpy pandas scikit-learn

2. Import Libraries

Start by importing the necessary libraries in your Python script or Jupyter notebook.

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

3. Load Dataset

For this experiment, you can use the Iris dataset, which is commonly used for classification tasks. You can load it directly from scikit-learn or use a CSV file.

Load Iris dataset from sklearn

from sklearn.datasets import load_iris

data = load_iris()

X = data.data # Features

y = data.target # Labels

If using a CSV file:

 # Load dataset from CSV file

data = pd.read_csv('your_dataset.csv')

X = data.iloc[:, :-1].values # Features (all columns except the last)

y = data.iloc[:, -1].values # Labels (last column)

4. Preprocess Data Split the dataset into training and testing sets.

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

5. Train Naive Bayes Classifier

Create an instance of the Gaussian Naive Bayes classifier and fit it to the training data.

python

nb_classifier = GaussianNB()

nb_classifier.fit(X_train, y_train)

6. Make Predictions

Use the trained classifier to make predictions on the test set.

y_pred = nb_classifier.predict(X_test)

7. Evaluate Performance

Calculate accuracy and generate a confusion matrix and classification report.

accuracy = accuracy_score(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)

class_report = classification_report(y_test, y_pred)

```
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", conf_matrix)
print("Classification Report:\n", class_report)
```

OUTPUT :

Conclusion :

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 04

(“Generation of Random Variables using Different Probability Distributions")

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS: M.Tech

SEMESTER: II

DATE OF SUBMISSION:

Title: Generation of Random Variables using Different Probability Distributions.

Objective:

To generate random variables following different probability distributions such as Uniform, Normal, and Exponential using computational methods.

Theory:

In pattern recognition**,** randomness is often used in simulations, model testing, and probabilistic algorithms. Computers typically generate pseudo-random numbers from a Uniform(0,1) distribution. These are then transformed to generate other distributions.

Key Methods:

1. Inverse Transform Method

If $U \sim Uniform(0,1)$, then:

$$X = F^{-1}(U)$$

produces a random variable with distribution F(x)F(x)F(x).

2. Normal Distribution (Gaussian)

Using Box-Muller method:

$$Z = \sqrt{-2\ln U_1}\cos(2\pi U_2)$$

3. Exponential Distribution

$$X = -\frac{1}{\lambda}\ln(U)$$

Algorithm

◆ **Uniform Distribution**

1. Generate random number U∈(0,1)
2. Output U

◆ **Exponential Distribution**

1. Generate U~Uniform(0,1)
2. Compute:

$$X = -\ln(U)/\lambda$$

◆ **Normal Distribution**

1. Generate two independent uniform numbers U1,U2
2. Compute:

$$Z = \sqrt{-2\ln U_1}\cos(2\pi U_2)$$

Procedure:

Step 1: Setup Environment

1. Open Python IDE (e.g., Jupyter Notebook / VS Code).
2. Import required libraries:
    - `random`
    - `math`
    - `matplotlib`

Step 2: Generate Uniform Random Variables

1. Use `random.random()` to generate numbers between 0 and 1.
2. Store at least 1000 values in a list.
3. Plot histogram to visualize distribution.

Step 3: Generate Exponential Random Variables

1. Generate a uniform random number U
2. Apply transformation: $X = -\ln(U)/\lambda$
3. Repeat for multiple samples (e.g., 1000 values).
4. Plot histogram.

Step 4: Generate Normal Random Variables

1. Generate two independent random numbers U1 & U2
2. Apply Box-Muller formula:

$$Z = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$

3. Generate sufficient values (1000).
4. Plot histogram.

◆ Step 5: Visualization

1. Plot histograms for:
    - Uniform distribution
    - Exponential distribution
    - Normal distribution
2. Compare their shapes.

Step 6: Analyze Results

1. Observe distribution patterns:
    - Uniform → flat
    - Exponential → right-skewed
    - Normal → bell-shaped
2. Verify correctness visually.

Program :

```
import random

import math

import matplotlib.pyplot as plt


# Uniform
```

```
uniform = [random.random() for _ in range(1000)]


# Exponential
exp = [-math.log(random.random()) for _ in range(1000)]


# Normal (Box-Muller)
normal = []
for _ in range(500):
    u1 = random.random()
    u2 = random.random()
    z1 = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
    z2 = math.sqrt(-2 * math.log(u1)) * math.sin(2 * math.pi * u2)
    normal.extend([z1, z2])
# Plot
plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.hist(uniform, bins=30)
plt.title("Uniform")
plt.subplot(1,3,2)
plt.hist(exp, bins=30)
plt.title("Exponential")
plt.subplot(1,3,3)
```

```
plt.hist(normal, bins=30)

plt.title("Normal")


plt.show()
```

OUTPUT :

Conclusion :

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 03

(Implementation and Study of Linear Perceptron Learning for Binary Classification)

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS:

SEMESTER: II

DATE OF SUBMISSION:

Title:

Implementation and Study of Linear Perceptron Learning for Binary Classification

Aim:

To understand and implement the Linear Perceptron algorithm for binary classification and study its performance on a linearly separable dataset. The experiment aims to demonstrate the perceptron's learning process and analyze its ability to classify data points.

Theory:

The **Linear Perceptron** is a type of supervised machine learning algorithm used for binary classification. It works by finding a linear hyperplane that separates two classes of data in a feature space. The perceptron updates its weights and bias based on misclassifications until it converges or reaches the maximum number of iterations.

The perceptron works based on the following decision rule:

y=sign(w·x+b)y = \text{sign}(w \cdot x + b)y=sign(w·x+b)

Where:

- yyy is the predicted class label (1 or -1).
- xxx is the input feature vector.
- www is the weight vector.
- bbb is the bias term.

The perceptron is trained using the **Perceptron Learning Rule**, which adjusts the weights whenever the model misclassifies a sample. The update rule is:

w←w+η(ytrue−ypred)xw \leftarrow w + \eta (y_{\text{true}} - y_{\text{pred}})
xw←w+η(ytrue−ypred)x

Where:

- η\etaη is the learning rate.
- ytruey_{\text{true}}ytrue is the true label.
- ypredy_{\text{pred}}ypred is the predicted label.
- xxx is the feature vector.

Procedure:

Step 1: Prepare the Dataset

For this experiment, we will use a simple linearly separable dataset. A good example is the **Iris dataset** (classifying Iris flowers into two classes based on features like petal length, petal width, etc.), but for simplicity, we will generate a synthetic dataset that is linearly separable.

Step 2: Initialize the Perceptron

We initialize the weight vector w and the bias b. These are often initialized to zeros or small random values. The learning rate η is also set.

Step 3: Perceptron Learning Rule

Iterate over the dataset, calculate the output for each sample, and update the weights whenever a misclassification occurs. The update rule is applied according to the formula mentioned earlier.

Step 4: Train the Perceptron

Perform the learning process for a specified number of epochs (iterations through the entire dataset). During each epoch, check for misclassified points, and update the weights accordingly.

Step 5: Test and Evaluate

After training, evaluate the perceptron's performance on the same or separate test set. Measure the accuracy and analyze the decision boundary.

Detailed Steps:

Step 1: Import Required Libraries

Start by importing necessary libraries such as NumPy for numerical operations.

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Generate a Simple Linearly Separable Dataset

We generate a synthetic dataset where the classes are separable by a straight line.

```
# Generate a simple linearly separable dataset
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2, n_informative=2,
n_redundant=0, n_classes=2, random_state=42)

# Visualize the dataset
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.title("Linearly Separable Dataset")
plt.show()
```

Step 3: Define the Perceptron Class

Create a class for the Perceptron algorithm with methods for fitting the model and making predictions.

```
class Perceptron:
    def __init__(self, learning_rate=0.1, n_iter=1000):
        self.learning_rate = learning_rate
        self.n_iter = n_iter
        self.weights = None
        self.bias = 0

    def fit(self, X, y):
        # Initialize weights to zeros
        self.weights = np.zeros(X.shape[1])

        # Training process
        for _ in range(self.n_iter):
            for xi, target in zip(X, y):
                update = self.learning_rate * (target - self.predict(xi))
                self.weights += update * xi
                self.bias += update

    def predict(self, X):
        return np.where(self.net_input(X) >= 0.0, 1, -1)

    def net_input(self, X):
        return np.dot(X, self.weights) + self.bias
```

Step 4: Train the Perceptron

Use the Perceptron model to train on the dataset.

```
# Initialize and train the perceptron
perceptron = Perceptron(learning_rate=0.1, n_iter=1000)
perceptron.fit(X, y)
```

Step 5: Plot the Decision Boundary

Visualize the decision boundary learned by the perceptron.

```
# Plot the decision boundary
xx, yy = np.meshgrid(np.linspace(X[:, 0].min(), X[:, 0].max(), 100),
                     np.linspace(X[:, 1].min(), X[:, 1].max(), 100))

Z = perceptron.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o',
cmap=plt.cm.Paired)
plt.title("Perceptron Decision Boundary")
plt.show()
```

Step 6: Evaluate the Model

After training, evaluate the perceptron on the dataset to calculate accuracy.

```
# Accuracy
y_pred = perceptron.predict(X)
accuracy = np.mean(y_pred == y)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

Conclusion:

The experiment demonstrates how the Linear Perceptron algorithm is used for binary classification. The perceptron learns to separate the two classes by finding an optimal linear boundary. The accuracy of the perceptron on a linearly separable dataset should be high, indicating that the perceptron can successfully classify the data.

Exercise 1: Implement Perceptron from Scratch

- **Task:** Write a Python class for the Perceptron algorithm from scratch. Implement the learning rule, update weights, and use a step function for classification.
- **Expected Outcome:** A fully functional Perceptron model that can classify linearly separable data.

Exercise 2: Visualize Perceptron Decision Boundary

- **Task:** Using the Perceptron class you created in Exercise 1, generate a 2D synthetic dataset (use `make_classification` from `sklearn` for simplicity). After training the Perceptron, visualize the decision boundary and the data points.
- **Expected Outcome:** A plot showing the decision boundary learned by the perceptron and how it classifies data.

Exercise 3: Experiment with Learning Rate

- **Task:** Train the Perceptron with different learning rates (e.g., 0.01, 0.1, 1) and observe how the model performance changes. Record the accuracy of the model after training.
- **Expected Outcome:** A comparison of how different learning rates affect the perceptron's convergence and accuracy.

Exercise 4: Implement Early Stopping in Perceptron

- **Task:** Modify the Perceptron code to implement early stopping. The training process should stop when no misclassifications occur during an entire pass over the training data.
- **Expected Outcome:** The Perceptron stops early if it converges before reaching the maximum number of iterations.

Exercise 5: Evaluate Perceptron on Real Dataset

- **Task:** Use the **Iris dataset** (from `sklearn.datasets`) and apply the Perceptron algorithm to classify the two classes of Iris (setosa and versicolor). Evaluate the model's performance on this dataset.
- **Expected Outcome:** A classification report showing the accuracy, precision, recall, and F1-score for the model.

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 02

(Exploring Mean and Covariance Statistical Foundations for Pattern Recognition)

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS:

SEMESTER: II

DATE OF SUBMISSION:

Aim: Exploring Mean and Covariance Statistical Foundations for Pattern Recognition.

A. Objective

Calculate the Maximum Likelihood (ML) estimate of the mean and covariance for given datasets.

Understand the relationship between mean, covariance, and data distribution.

Visualize the results to draw conclusions about data characteristics.

B. Materials Required

- Programming environment (e.g., MATLAB or Python).
- Sample datasets (can be generated or sourced from existing datasets).
- Visualization tools (e.g., plotting libraries).

C. Procedure

1. create two classes of data with different means and covariance's
2. Implement functions to calculate the mean and covariance
3. Use ML estimation to compute unbiased estimates
4. Plot the datasets along with their estimated means and contours representing their covariances

Detail Codes:

1. **Generate a dataset with known parameters**:
   For instance, create two classes of data with different means and covariance's:

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters for Class 1
mu1 = np.array([1, 1])
sigma1 = np.array([[1, 0], [0, 1]])
class1 = np.random.multivariate_normal(mu1, sigma1, 10000)

# Parameters for Class 2
```

```
mu2 = np.array([4, 4])
sigma2 = np.array([[4, 0], [0, 16]])
class2 = np.random.multivariate_normal(mu2, sigma2, 10000)

# Combine classes
data = np.vstack((class1, class2))
labels = np.array([0]*10000 + [1]*10000)
```

2. Calculating Mean and Covariance

```
def calculate_mean(data):
    return np.mean(data, axis=0)

def calculate_covariance(data):
    return np.cov(data, rowvar=False)

mean_data = calculate_mean(data)
covariance_data = calculate_covariance(data)
```

3. Maximum Likelihood Estimation

```
def ml_estimate(data):
    n_samples = data.shape[0]
    mean_estimate = calculate_mean(data)
    covariance_estimate = (1 / (n_samples - 1)) * calculate_covariance(data)
    return mean_estimate, covariance_estimate

mu_estimated, sigma_estimated = ml_estimate(data)
```

4. Visualization : Plot the datasets along with their estimated means and contours representing their covariances:

```
plt.figure(figsize=(10, 6))
plt.scatter(class1[:, 0], class1[:, 1], alpha=0.5, label='Class 1')
plt.scatter(class2[:, 0], class2[:, 1], alpha=0.5, label='Class 2')

# Plotting means
plt.scatter(mu_estimated[0], mu_estimated[1], color='red', label='Estimated
Mean')

# Contours for covariance
def plot_covariance_ellipse(mean, cov):
    v, w = np.linalg.eigh(cov)
    u = w[0] / np.linalg.norm(w[0])
    angle = np.arctan(u[1] / u[0])
    angle = np.degrees(angle)
    ell = plt.matplotlib.patches.Ellipse(mean, v[0]*2, v[1]*2,
                                         angle=angle,
```

```
                                color='green', alpha=0.5)
    plt.gca().add_patch(ell)

plot_covariance_ellipse(mu_estimated, sigma_estimated)

plt.title('Mean and Covariance Visualization')
plt.legend()
plt.grid()
plt.show()
```

Analysis: After visualizing the results do the analysis and write in details here..

Conclusion

This experiment will provide insights into how mean and covariance can be used to characterize data distributions in pattern recognition tasks.

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

PATTERN RECOGNITION LAB

LABWORK - 01

(Feature Representation in Pattern Recognition)

COURSE NAME: PATTERN RECOGNITION LAB

COURSE CODE: CSEMT4209

DEPARTMENT: Amity School of Engineering & Technology

FACULTY NAME: Dr. Sarang Maruti Patil

SUBMITTED BY

STUDENT NAME:

ENROLLMENT NUMBER:

CLASS: DIV A

SEMESTER: 7th

DATE OF SUBMISSION:

A. Objective

The objective of this experiment is to explore how different feature representations affect the classification performance of a machine learning model. We will use the popular Iris dataset, which contains measurements of iris flowers and is commonly used for classification tasks.

- **Dataset**

The Iris dataset consists of 150 samples of iris flowers, each with four features:

Sepal Length

Sepal Width

Petal Length

Petal Width

The target variable is the species of the iris flower, which can be one of three classes: Setosa, Versicolor, or Virginica.

Procedure

1. Load the Dataset: Use the sklearn library to load the Iris dataset.
2. Data Pre-processing: Split the dataset into training and testing sets.
3. Feature Representation: Experiment with different feature representations:

   Use all four features.
   Use only two features (e.g., Sepal Length and Sepal Width).
   Create new features through polynomial combinations.
4. Model Training: Train a classification model (e.g., Decision Tree Classifier) on each feature set.
5. Model Evaluation: Evaluate the model's performance using accuracy as a metric.
6. Compare Results: Analyze how different feature sets impact classification accuracy.

Code Implementation

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable (species)

# Step 1: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Function to evaluate model performance
def evaluate_model(X_train, X_test, y_train, y_test):
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    return accuracy

# Step 2: Evaluate using all four features
accuracy_all_features = evaluate_model(X_train, X_test, y_train, y_test)

# Step 3: Evaluate using only two features (Sepal Length and Sepal Width)
X_train_2d = X_train[:, :2]  # Select first two features
X_test_2d = X_test[:, :2]
accuracy_2d_features = evaluate_model(X_train_2d, X_test_2d, y_train, y_test)

# Step 4: Create polynomial features (Sepal Length^2 and Sepal Width^2)
X_train_poly = np.column_stack((X_train[:, 0]**2, X_train[:, 1]**2))
X_test_poly = np.column_stack((X_test[:, 0]**2, X_test[:, 1]**2))
accuracy_poly_features = evaluate_model(X_train_poly, X_test_poly, y_train, y_test)

# Step 5: Print results
print(f"Accuracy with all features: {accuracy_all_features:.4f}")
```

```
print(f"Accuracy with 2D features (Sepal Length & Width): {accuracy_2d_features:.4f}")
print(f"Accuracy with polynomial features (Sepal Length^2 & Width^2): {accuracy_poly_features:.4f}")

# Optional: Visualize results
labels = ['All Features', '2D Features', 'Polynomial Features']
accuracies = [accuracy_all_features, accuracy_2d_features, accuracy_poly_features]

plt.bar(labels, accuracies)
plt.ylabel('Accuracy')
plt.title('Model Accuracy for Different Feature Representations')
plt.ylim(0, 1)
plt.show()
```

Explanation of the Code

1. **Import Libraries**: The necessary libraries for data manipulation and machine learning are imported.
2. **Load Dataset**: The Iris dataset is loaded using `sklearn.datasets`.
3. **Data Splitting**: The dataset is split into training and testing sets using `train_test_split`.
4. **Model Evaluation Function**: A function `evaluate_model` is defined to train and evaluate the Decision Tree Classifier.
5. **Feature Representation Experiments**:
   - All four features are used for training and evaluation.
   - Only Sepal Length and Sepal Width are selected for a second evaluation.
   - Polynomial features are created by squaring Sepal Length and Sepal Width for a third evaluation.
6. **Results printing and Visualization**: The accuracies are printed out and visualized using a bar chart.

Conclusion:

This experiment allows us to observe how different feature representations can impact the performance of a classification model. We can further extend this experiment by exploring other classifiers or more complex feature engineering techniques!